

DATA203 Foundational Python (Prof. Maull) / Fall 2026 / HW1

Points Possible	Due Date	Time Commitment (estimated)
15	Wednesday, September 30 @ midnight	<i>up to 15 hours</i>

- **GRADING RUBRIC:** Grading will be aligned with the completeness of the objectives.

Your solution is evaluated on:

1. whether your solution correctly answers the question(s) being asked,
2. using the Python language as required in the question(s).

Your solution is **NOT** evaluated on:

1. the elegant use of Python,
2. optimal or high performance Python code unless *explicitly noted* in the question(s).

- **INDEPENDENT WORK:** Copying, cheating, plagiarism and academic dishonesty *are not tolerated* by University or course policy. Please see the syllabus for the full departmental and University statement on the academic code of honor.

- **LARGE LANGUAGE MODEL (LLM) / ARTIFICIAL INTELLIGENCE/LLM (AI) POLICY:**

LLMs / Artificial Intelligence technologies such as Claude, Gemini, Grok, etc are **NOT** to be used in the production of your final homework solutions.

Such technologies offer important capabilities for your *learning* journey, but they are not appropriate in this course for the production of your solutions.

You *may* use such technologies to *learn* more about specific Python concepts or to help clarify your knowledge about the details of Python syntax, modules, techniques or other programming principles.

Final solutions found to be the *partial or entire product* of LLM/AI use **will receive 0 points**, without clarification, resubmission or correction.

OBJECTIVES

- UNGRADED PRACTICE: Explore JupyterHub Python *shell* commands inside cells
- PART 1: Practice loading lists and dictionaries to/from JSON and YAML.
- PART 2: Compare dictionary data using loops.
- PART 3: Compare data using Python sets.
- PART 4: Align data into a single dictionary.
- PART 5: Create the 1990s ONLY golden era merged datasets.

WHAT TO TURN IN

You are being encouraged to turn the assignment in using the provided Jupyter Notebook. To do so, make a directory in your Lab environment called `homework/hw1`. Put all of your files in that directory. Then zip or tar that directory, rename it with your name as the first part of the filename (e.g. `maull_hw1_files.zip`, `maull_hw1_files.tar.gz`), then download it to your local machine, then upload the `.zip` to Canvas.

If you do not know how to do this, please ask, or visit one of the many tutorials out there on the basics of using zip in Linux.

If you choose not to use the provided notebook, you will still need to turn in a `.ipynb` Jupyter Notebook and corresponding files according to the instructions in this homework.

ASSIGNMENT TASKS

(0%) UNGRADED PRACTICE: Explore JupyterHub Python *shell* commands inside cells

In the last time we learned to run the terminal console commands in JupyterLab, which is a great way to perform command-line tasks and is an essential tool for basic scripting that is part of a data scientist's toolkit. Last time we used a terminal console in the lab environment this time we familiarize ourselves with Jupyter *shell* escape commands **within** a notebook.

Study:

- [Python Data Science Handbook: IPython and Shell Commands](#)

for full documentation on *shell* ... they are **very** useful!

\$ Task: 0.1 Use Jupyter *shell* commands to perform the same commands as last time.

Basic file operations go a long way to understand the way Linux works. In this part, you will understand folders, files and making revisions to a file. These files will be visible within Jupyter, which makes moving from one platform to another seamless. We will create a folder, file and make edits.

- type `!mkdir your_folder_name` to create a folder in filesystem *in the current folder where you are*
- create a file by type `touch README.md` the touch command creates a file if it does not already exist, otherwise it will change the timestamp of that file when it is "touched"
- edit the file in Jupyter with the text editor
- to see the contents of your file typing `!cat README.md`

\$ Task: 0.2 Use *shell* command `wget` to quickly obtain remote files in Linux

As before get a remote file this time it will be from the [Internet Archive](#):

- in a cell type `!wget https://ia801306.us.archive.org/15/items/fouraddressesats00howa/fouraddressesats00howa`
- execute the cell
- verify the file was retrieved by opening it

(20%) PART 1: Practice loading lists and dictionaries to/from JSON and YAML.

For all of the homework you have been provided a starter notebook for use, which will greatly enhance your ability to complete the assignment. See that in the Github folder and find the notebook `hw1/hw1_starter.ipynb`:

- https://github.com/kmhuads/f26_data203/tree/main/hw1/hw1_starter.ipynb

Look at this file and see what is in it – use it since in the notebook are some scaffolding code. You will need to study it and use it in the solutions being asked.

If you used `git clone` to get the homework from last time, you should go back to the same folder and in the terminal type `git pull`. This will load the latest files onto your Jupyter. Use the starter notebook to write the Python code required for the assignment.

DATASET BACKGROUND: HIP HOP'S GREATEST ALBUMS

Music ratings and rankings are highly subjective. We all have our preferences and interests and very rarely can we **all** agree on **all** things music.

Whether you are a historian or not, here in the first quarter of the 21st century, we are now able to look back into music with a lens towards the "golden era" of Hip Hop, roughly beginning in the late 1980s through the 90s – depending on which ethnomusicologist you consult. We would thus expect a large number of any "Greatest Albums" list to come from this 90s.

As the genre has evolved and time elapsed, we are seeing more and more "bests" and "greatest" lists. In this homework, we are going to use our knowledge of Python to explore two "Top 25 Best Albums of Hip Hop" lists found online.

We are **not** going to assert our own bias onto either of these lists, rather, explore them as they are and leave our opinions, perhaps, for a class discussion board should there be personal controversy over what we find.

For fun while you work through the first part of this homework, if you would like to listen to a starter playlist of 90s classics (ft. Heavy D, Queen Latifah, Digital Underground, MC Lyte, Onyx, ATCQ and gems from many others) check out this nice mix:

- DJ Afrosia [90s Hip-Hop | Golden Era Classics](#)
- or Ad free:

PYTHON BACKGROUND

In this part we are going to work with dictionaries and loops again. To do so, we are going to learn about two common file types that you will run across in Python (and other languages).

The first file type is called JSON or “JavaScript Object Notation” and is one of the most ubiquitous file formats today. It allows you to store data of all kinds, but in our particular case, as we will find out, it is well suited for mapping data, key-value paired data or what we now know to be dictionaries in Python.

You will often know a file is a JSON file because it ends in the file extension `.json`.

You will want to pay close attention to this because being able to store and retrieve dictionaries and key-value paired data is greatly simplified when using JSON.

The other file type is called YAML or “YAML Ain’t Markup Language”. YAML is commonly identified by the `.yaml` extension, and from a technical perspective is a more robust version of JSON. It is distinguished by a more precise set of rules for indentation within files (much like Python’s indentation rules), and arguably provides more readable and accessible syntax for beginners. Indeed, many non-technical data entry approaches using YAML can be very natural and intelligible to non-experts from many disciplines. Furthermore, as we will see, YAML and JSON can be programmatically converted from one to another quite easily in Python.

You are responsible for learning about JSON and YAML from these resources:

- YAML [module documentation on pyyaml.org](#)
- JSON [module documentation on Python.org](#)

Of course, you may look at other resources to expand your understanding.

\$ Task: 1.1 Explore the starter notebook.

You will notice two variables: `ds1` and `ds2`. Study them to complete the tasks that follow.

\$ Task: 1.2 Explain in your own words the difference between the structures used to hold `ds1` and `ds2`.

Be sure to include:

- what data types are used in each (e.g. lists, dictionaries, etc.),
- what actual data they hold (e.g. titles, names, etc), and
- how are they similar or different.

Your explanation should be suitable for someone who **does not know** Python, but be descriptive enough so that they have enough detail to understand what is going on.

\$ Task: 1.3 Load the full datasets from the included YAML and JSON using the required functions into variables `ds1_yaml` and `ds2_json`.

You can load YAML files with the following Python. First you must load or import the correct modules:

```
import yaml
```

After this is executed in a cell, you can study the official documentation for [`yaml.safe\_load\(\)`](#).

For example, if there is a file called `"data.yaml"`, then

```
yaml.safe_load(open("data.yaml"))
```

will load the file into a Python dictionary.

Do the same with the included JSON file:

```
import json
```

and you will use [`json.load\(\)`](#), such that if there is a file `"data.json"`, then,

```
json.load(open("data.json"))
```

will load the file into a Python dictionary.

YOUR CODE MUST ASSIGN TWO VARIABLES: `ds1_yaml` and `ds2_json`, respectively using the `safe_load()` and `load()` functions above AND the files you will need load are named accordingly in the same folder as the starter notebook (not `data.json` or `data.yaml` those are just example names).

(20%) PART 2: Compare dictionary data using loops.

We have now learned to load data into variables from files – soon we will learn a lot more about files and Python I/O (Input/Output) but for now, the knowledge we have obtained will suffice.

You will now use the variables in PART 1 to complete the tasks for PART 2.

\$ Task: 2.1 Write a for loop to print the data in `ds1_yaml`.

Your output must be EXACTLY in the form:

```
{ranking} {artist} > {title} ({year})
```

So that it looks like:

```
#1 Nas Illmatic > (1994)
#2 Kendrick Lamar > To Pimp A Butterfly (2015)
#3 Wu-Tang Clan > Enter The Wu-Tang (1993)
...
```

For ALL 25 albums in `ds1_yaml`.

\$ Task: 2.2 Write a for loop over the data in `ds2_json` to display just the albums that came out between 1992-1997 (inclusive).

You will use a technique similar to the first task, but this time you must use an `if` statement to determine if the year is 1992 - 1997 (**REMEMBER: 1992 and 1997 ARE INCLUDED!**)

Your output will look something like:

```
#2 Nas > Illmatic (1994)
#6 Wu-Tang Clan > Enter The Wu Tang (1993)
#7 A Tribe Called Quest > Midnight Marauders (1993)
...
```

\$ Task: 2.3 Write a for loop to find the *mean* (average) year of all albums from EACH list.

Use the Python built-in `round()` to round the final average.

Your output will look like:

```
Average album year for HHGA.com: 1955
Average album year for SL.com: 1908
```

Obviously your numbers will be in the correct decade!

- You can use list comprehension or any other `for` looping technique you would like to use.
- You may also find the Python `sum()` built-in very valuable – and don't overthink it, there are only 25 items in each list.

(20%) PART 3: Compare data using Python sets.

You may have already noticed that the two datasets have a lot in common, but they also have quite a few differences.

For example, on close manual inspection of the top 5, the two lists only have Nas' *Illmatic* (1994) and Madvillain's *Madvillainy* (2004) in common.

We are going to dig deeper it this discrepancy.

\$ Task: 3.1 Write the code that compares the album *artists* from both lists showing the ones they have in common.

You MUST use Python sets to complete this part.

This should be very straightforward, but remember:

1. you will need to loop over all artists and place them into a sets,
2. you will then need to compare these two sets using `intersection()`.

Consult with the official documentation on Python sets here:

- Python [official sets tutorial documentation](#).

Your output will look like:

```
The two lists have the following artists in common:
```

```
* James Brown
* Prince
* Force MDs
...
```

\$ Task: 3.2 Write the code that compares the album *titles* from both lists showing the one's they have in common.

The output for this part of the assignment will look like this:

```
The two lists have the following albums in common:
```

```
* James Brown > Payback! (1972)
* Prince > Under a Cherry Moon (1986)
* Force MDs > Touch and Go (1987)
...
```

Note you **MUST** print the entirety of the artist, album name and year **as shown**.

This will be done with a loop looking up the matching title from either list of your choosing.

(20%) PART 4: Align data into a single dictionary.

Now we are learning that the structural differences between the two datasets is causing a lot more code to be written since the two datasets do not *align*. That is to say that one list has keys which represent the ranking, while the other the ranking is part of the dictionary structure for each album.

Neither structure is “correct”, they’re just “different”.

We’re going to align them so that we have a *single* dictionary that holds them both and has the same structure for all data.

\$ Task: 4.1 Write the code that creates a single dictionary called `aligned_full_data` holding both lists.

The dictionary you will create will the following properties:

1. the dictionary will have numeric (integer keys) 1 through 25 representing the ranking for an album,
2. each ranking key will have two sub-keys: `hhga` and `sl` (for “hip hop golden era” and “shortlist”, respectively), for example the ranking key 1 will have the #1 ranked “hip hop golden era” (from the json file) album under the `hhga` and the #1 ranked “shortlist” album (from the yaml file),
3. each sub key will have three sub-keys: `title`, `artist`, `year` which contain the obvious information about the corresponding `hhga` and `sl` album.

Your final dictionary structure will look like:

```
1: {'hhga': {
    'artist': 'Public Enemy',
    'title': 'It Takes A Nation Of Millions To Hold Us Back',
    'year': 1988
  },
  'sl': {
    'artist': 'Nas',
```

```

        'title': 'Illmatic',
        'year': 1994
    },
},
2: {'hhga': {
    'artist': 'Nas',
    'title': 'Illmatic',
    'year': 1994
},
'sl': {
    'artist': 'Kendrick Lamar',
    'title': 'To Pimp A Butterfly',
    'year': 2015
}
},
3: {'hhga': {
    'artist': 'A Tribe Called Quest',
    'title': 'The Low End Theory',
    'year': 1991
},
'sl': {
    'artist': 'Wu-Tang Clan',
    'title': 'Enter The Wu-Tang',
    'year': 1993
}
},
...

```

Don't forget to store this new dictionary in a variable named "aligned_full_data".

(20%) PART 5: Create the 1990s ONLY golden era merged datasets.

Here we are, now at the end of the dataset investigation up to this point. We have learned that there is a lot of variation in the data, and notably we find that one list has more albums from the 80s and the other list more albums from the 00s.

However, when we look at the 90s (1990-1999) there is a tie – they *both* have 14 albums on their lists, but only 4 of them are in agreement! Arguably, the 1990s represents a wide diversity of golden era artists, themes, regions and styles so even the “best of” lists do not have full agreement on which albums are the “best” – perhaps pointing to why the “golden” era is so important to hip hop.

\$ Task: 5.1 Write the code that merges all the 1990s albums from *both* lists. The list should have NO duplicates.

Your final output should be a list of dictionaries with all the 90s albums from of both lists and will look something like this:

```

[
    {'artist': 'A Tribe Called Quest',
     'title': 'The Low End Theory',
     'year': 1991},
    {'artist': 'Wu-Tang Clan',
     'title': 'Enter The Wu Tang',
     'year': 1993},
    {'artist': 'Pete Rock & CL Smooth',
     'title': 'Mecca And The Soul Brother',
     'year': 1992},
    ...
]

```

Note that you do not need to carry forward *which* list the album came from, nor it's ranking. You need only make sure if there are duplicates (which we know there are 4), there is only **one** of the duplicates represented in the final list.